[HackerNotes Ep. 172] Critical Guide to Code Review

In this week's episode we're talking about code review! Here you'll find our full guide to code review with our favourite techniques and methodology, from mapping auth on every route before reading a single handler, to tracing taint flows, sniffing for anomalies and chaining low-severity bugs into crits.

Yujilik
April 30, 2026

Hacker TL;DR

1. **Map auth requirement on every route before reading a single handler:** The Grafana CVE-2020-13379 chain (25+ crits) came from one anomaly: every route had reqSignedIn except /avatar/:hash. Annotate routes with their auth middleware in a table, sort by exposure, and the unauth rows are your targets, pre-auth bugs scale across programs and pay better.
2. **Framework wrappers are the real sinks, don’t grep only for eval()** total.js's U.set() reaches new Function() internally with a bypassable blacklist. Ruby object.send(params[:method]) invokes anything on the object including Kernel#system. Spring controllers returning user input as a view name get it interpreted as SpEL. The dangerous behaviour is one or two layers inside framework code, never visible at the call site, sink lists must be built per framework, not per language primitive.
3. **Parser differentials:** when the security layer and backend interpret input differently, the security layer can be broken. When a WAF or auth gateway parses one way and the backend parses another, the disagreement between them is the bypass you need to chase.
4. **Custom sanitizers fail in five predictable ways, run the checklist on every one**
    - Case-insensitive?
    - Global (/g or replaceAll)?
    - Recursive (one pass leaves ..../// → ../)?

- Complete (a SQLi filter without UNION is still exploitable)?
- Consistent across all routes?

Dev-written security functions are almost always bypassable on at least one of these, and this checklist gets you to the bug.

5. **"Sniff for blood": anomalies are bugs in disguise, chase them immediately**
   The one endpoint missing auth when every other has it, or the URL-fetcher that follows redirects, or the custom sanitizer that diverges from the standard library... When something looks off, the developer lost control of something. Open a scratch file, write down "what's the worst case if I fully control this?" and work backward.

Types of Code Review

> **TLDR:** Three modes. Use Baseline for new targets, Diff-Based after patches drop, Taint Analysis as your mental framework for tracing data.

Baseline Review

Full audit of an entire codebase from scratch.

You build a map: architecture, routes, auth model, dependencies, data flows.

Use this when:

- First time on a target
- You just got source code via Docker, decompilation, trial, etc.
- Prepping for a live hacking event

Goal of a baseline session: **notes and a map**, not necessarily findings. The map makes everything else faster.

Diff-Based Review

Look only at what changed: a patch, a new feature, a version bump. Ask: does this change open new attack surface? Does it weaken an existing control?

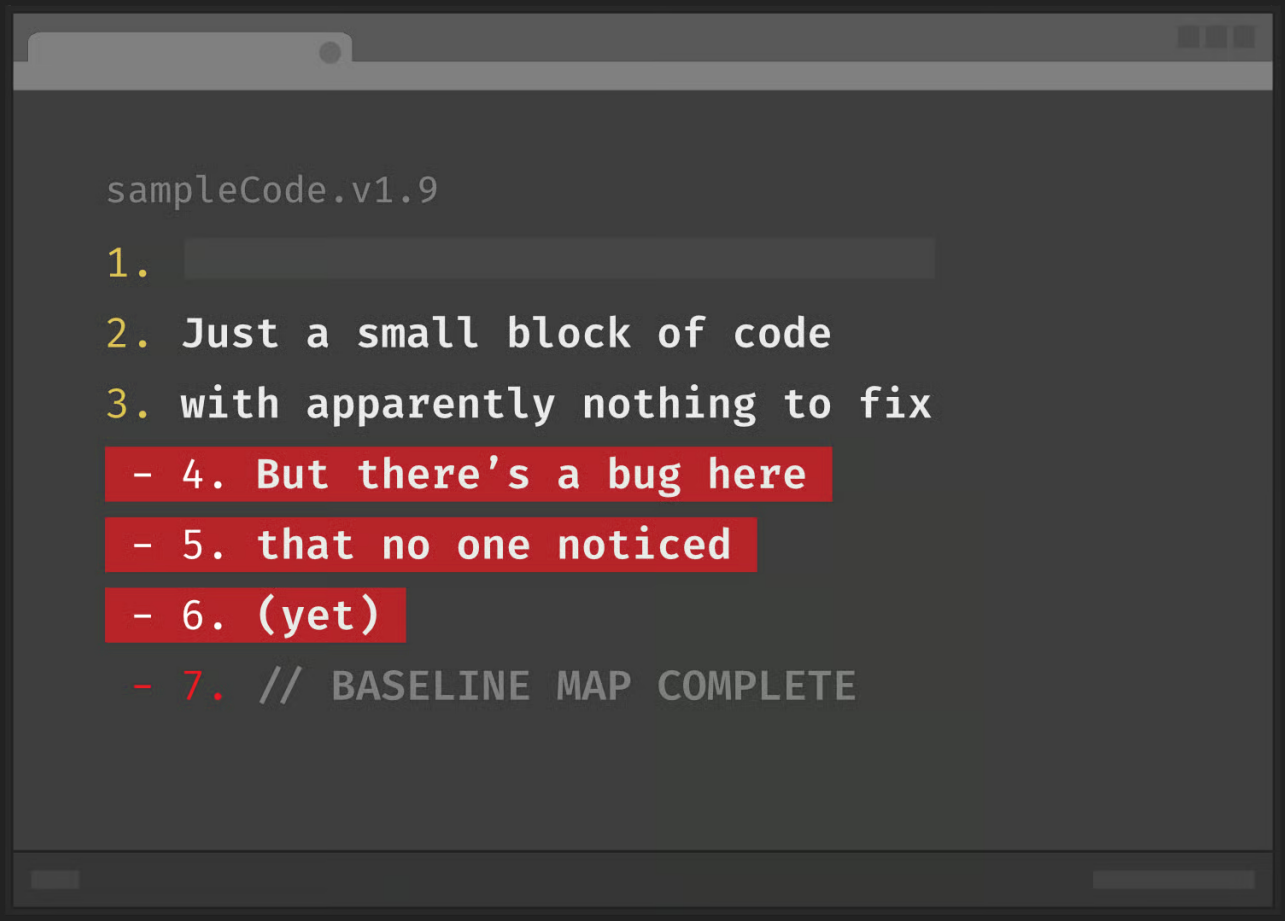
Use this when:

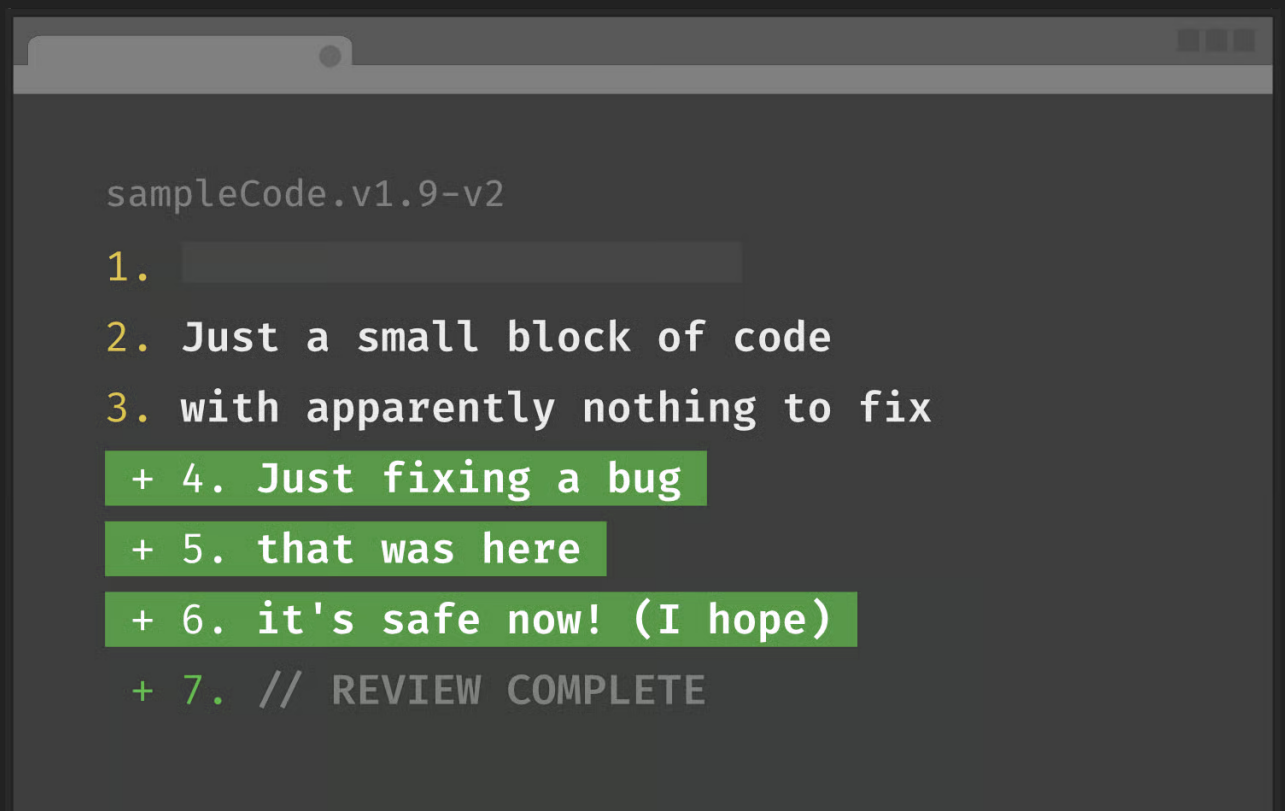
- A CVE patch drops for software you know: the fix often reveals the bug pattern

and where to look for variants

- A program adds new scope or announces a new feature
- Monitoring a target's public commits for regressions

Always expand the diff: a few extra lines of context can change whether a change is safe or not. And patches do not always fix the entire codebase, look for the same pattern elsewhere.





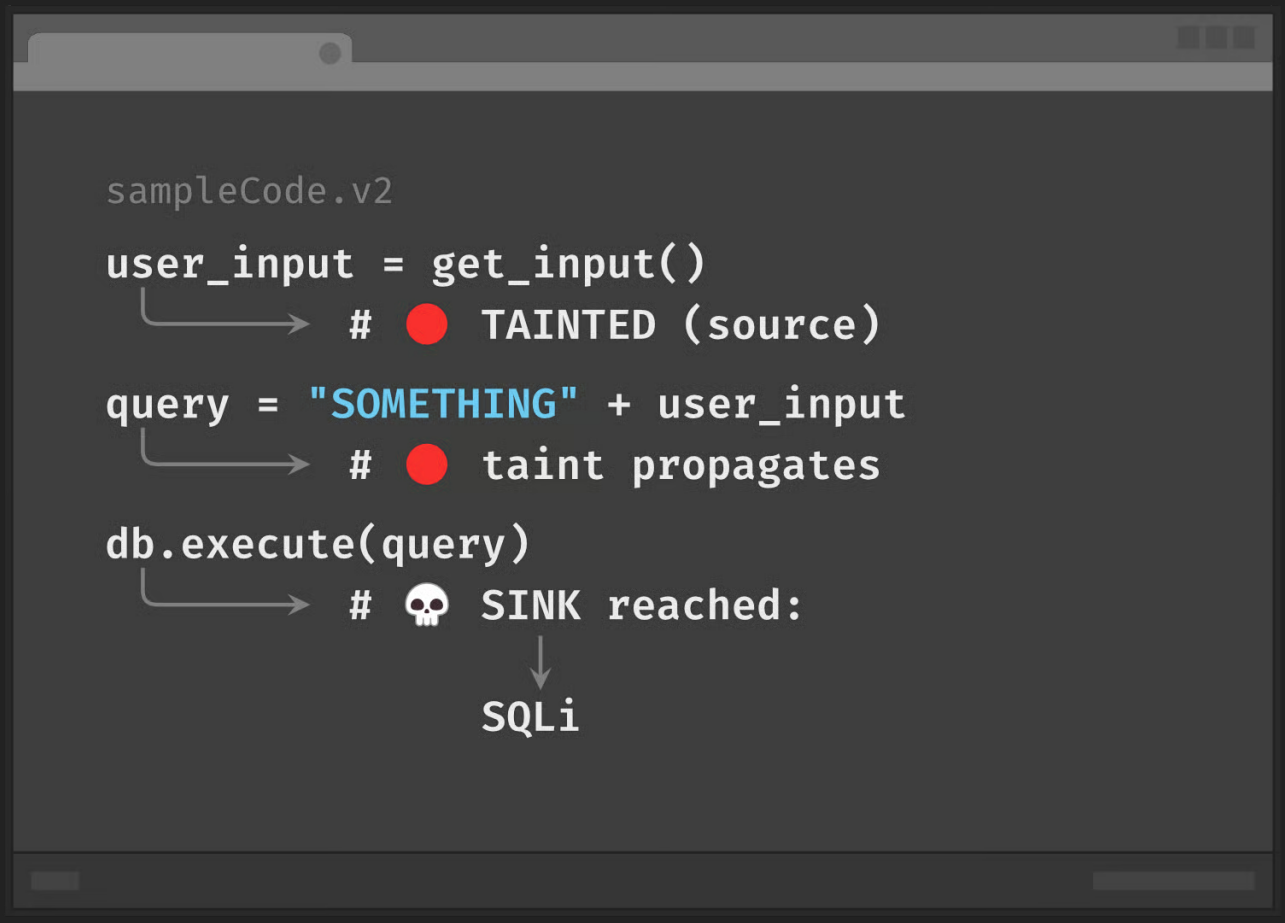
Taint Analysis

A formal model for thinking about how data flows through code. Every piece of user-supplied data is “tainted.” Track it through four components:

Sources: where tainted data enters:

- `request.getParameter("id")` (Java)
- `$_GET['input']` (PHP)
- `req.body.email` (Node.js)
- Headers, cookies, file uploads, WebSocket payloads

Propagators: pass taint along without cleaning it:



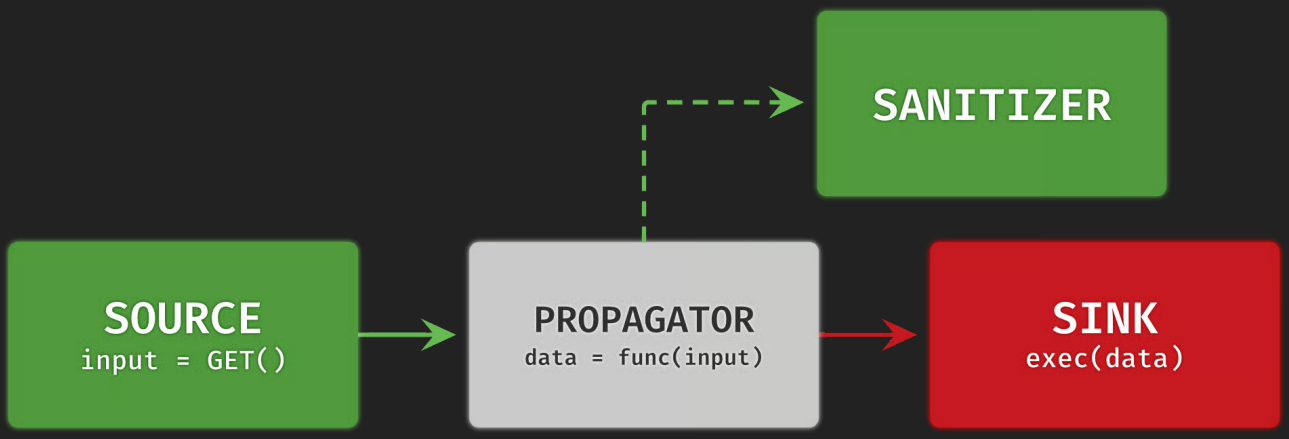
Assignments, function calls that return the input, array operations = propagators.

Sanitizers: functions that clean tainted data. The key word is *effective*. A sanitizer that can be bypassed doesn’t count.

Sinks: where tainted data causes harm. A vuln is confirmed when taint flows from source

→ sink with no effective sanitizer in between:

Instead of “is there a sanitizer?” the real question is “is the sanitizer effective? / is it in the right place?”



Part 1: Foundations

1.1 Sources and Sinks

TLDR: Source = where attacker data enters. Sink = dangerous function it could reach. Vulnerability = source connects to sink without a real fix in between.

Vulnerability	Example Sinks
SQLi	`execute()`, `query()`, raw SQL string concat
XSS	`innerHTML`, `document.write()`, raw template output
RCE	`eval()`, `exec()`, `system()`, `Runtime.exec()`
Path Traversal	`open()`, `readFile()`, `fopen()`, `new File()`
XXE	`DocumentBuilder.parse()`, `SAXParser.parse()` without restrictions
SSRF	`fetch()`, `curl_exec()`, `http.get()`, any URL-fetching library
Deserialization	`unserialize()` (PHP), `ObjectInputStream.readObject()` (Java), `pickle.loads()` (Python)

First pass habit: Make two lists, list all places user data enters and all dangerous functions you can find. Then ask: can these connect?

1.2 First Pass Through a Codebase

> **TLDR:** Don't dive into rabbit holes. Understand the structure first. Map before you hunt.

1. **Understand the structure:** What framework? Where do routes, controllers, and models live?
2. **Build your sinks list:** Every language has dangerous functions. PHP: `eval`, `system`, `exec`, `preg_replace /e`, `unserialize`. Java: `Runtime.exec`, `ProcessBuilder`, `DocumentBuilder.parse`.
   Watch out for framework wrappers: In modern apps you often won't see raw dangerous functions but rather a framework method that calls them internally. If you only grep for `eval()` or `exec()`, you'll miss most of what's actually exploitable. Understand what the framework's own functions do under the hood. Some examples:
   - **total.js `U.set()` / `U.get()`:** looks like a generic object utility but internally uses `new Function()` to build property accessors, making it a code injection sink. The framework added a regex blacklist to block dangerous input but the blacklist tests the raw string while `new Function()` interprets hex escapes at runtime. So `(` gets blocked, but `\x28` passes right through and becomes `(` inside the function. Tagged template literals handle the rest letting you *invoke functions without parentheses.*
   - **Ruby `object.send(params[:method])`:** `send` is a standard Ruby method for calling methods by name. If the method name comes from user input, the attacker can invoke anything on the object including `eval`, `exit`, and `system` via the Kernel module. *Documented in Ruby's own security guide* as explicitly dangerous.
   - **Spring Thymeleaf view name injection:** a Spring controller that returns user input as a view name string will have it interpreted as a Thymeleaf/SpEL expression by the framework. The "sink" is the return value of the controller method, not any obviously dangerous function call. Documented by *Veracode*.

   The pattern across all three: the dangerous behavior is one or two layers inside framework code, not visible at the call site. Build your sink list for the specific framework, not just the language primitives.
3. **Map routes first:** Find the routing config before reading any handler. `web.xml` (Java), `routes.rb` (Rails), `api.go` (Go), Express route files, `urls.py` (Django), etc.

4. **Take good notes first:** When you see a custom `sanitize()` call, note it and keep moving. Don't fall into early rabbit holes.
5. **Watch for developer mistakes:** Hardcoded credentials, second-order vulnerabilities (input stored and later executed), custom crypto.

1.3 Hardcoded Credentials and Secrets

> **TLDR:** Look for secrets baked into source files. Common and high impact.

Database passwords, AWS keys, JWT secrets, internal API tokens. They often grant immediate access to something, with no further exploitation needed.

Scan for: `password =`, `secret =`, `api_key =`, `token =`, `aws_access_key`, `-- BEGIN`.

Also check: `.env` files committed to version control, Docker environment variables in build history, and all Docker image layers, secrets "deleted" from layers still exist in earlier ones.

1.4 Custom Security Functions

> **TLDR:** Dev-written sanitizers are almost always bypassable. These are prime targets.

When you see a homemade security function, run through this:

- **Case-insensitive?** A blocklist for `SELECT` misses `select`. Look for `/i` flag or `.toLowerCase()`.
- **Global replacement?** JS `.replace("x", "")` only removes the *first* match. Needs `/g` flag or `.replaceAll()`.
- **Recursive?** Removing `../` once leaves `....//` → `../`
- **Complete?** A SQLi filter missing `UNION` is still exploitable. Check what's absent.
- **Applied consistently?** A sanitizer only called on some routes creates gaps.

1.5 Source-First vs. Sink-First

> **TLDR:** Source-first = start at inputs, trace forward. Sink-first = start at dangerous functions, trace backward. Use both.

Source-first: Start at entry points, route handlers, API endpoints, form inputs. Then follow the data through the app. Slower, but you build a real picture of how the app works. Best for finding business logic bugs and anything that requires understanding context.

Sink-first: Search for dangerous functions first (`eval()`, `Runtime.exec()`, raw SQL concat, etc.) and trace backward to see if user input can reach them. Fast way to check whether obvious high-severity classes like RCE, SQLi, or XXE exist at all. The downside: finding a sink doesn't mean you can reach it. It might only be called after auth checks pass (so you'd need a valid account), or it might be dead code that never runs in production. Both mean time wasted tracing something that goes nowhere.

Recommendation: Source-first as your default, you'll understand the app and catch more. Run a sink-first sweep at the end to make sure you didn't miss anything obvious.

Part 2: Getting Source Code

> **TLDR:** No source code, no review. There are many ways to get it even for paid, closed-source enterprise software.

2.1 JavaScript Source Maps

Source maps map minified JS back to the original source. A JS file declares one at the bottom:

```
//# sourceMappingURL=app.js.map
```

Devs often strip that line but forget to delete the actual `.map` file. Test by appending `.map` to any JS URL:

```
https://target.com/static/js/app.js.map
```

Browsers also auto-load them in the Sources tab of DevTools, check there first.

Extract the original source tree:

```
npx restore-source-tree app.js.map
```

This recreates the full directory of `.ts`, `.jsx`, or `.js` files, same as having the private repo.

What to look for: Hidden API endpoints, commented-out debug routes, internal service URLs, auth logic.

2.2 Exposed `.git` Directories

If a server publicly serves its `.git` folder, the entire repo history is downloadable. Test:

```
https://target.com/.git/config
https://target.com/.git/HEAD
```

If those return content (not 404), pull everything with `git-dumper`:

```
git-dumper https://target.com/.git/ ./output-dir
cd output-dir && git checkout .
```

If the server blocks some paths, try GitTools Extractor:

```
python3 extractor.py https://target.com/.git/ ./output-dir
```

Bonus: Full history includes deleted files. Old commits often have secrets. Run `trufflehog` or `gitleaks` to scan all commits automatically.

2.3 Docker Hub and Container Registries

Many enterprise vendors push Docker images publicly and sometimes even licensed software. Search `hub.docker.com`:

```
docker pull <vendor>/<image>:<tag>
docker run -it --entrypoint /bin/bash <image>:<tag>
```

Docker images are layered, giving you more angles beyond just a shell:

See the build history (often reveals secrets and paths):

```
docker history --no-trunc <image>:<tag>
```

Explore layers interactively with `dive`:

```
dive <image>:<tag>
```

Export and unpack raw filesystem:

```
docker save <image>:<tag> -o image.tar
tar -xf image.tar  # extract layer.tar files inside too
```

Automate it:

```
python3 docker-image-extract.py <image>:<tag>
```

Reconstruct the Dockerfile from history using `dedockify` or `dockerfile-from-image`.

2.4 Cloud Marketplaces

AWS, Azure, and GCP Marketplaces offer trial licenses for software that normally requires a sales call. Spin up a trial, SSH in, and pull application files from the filesystem.

2.5 GitHub Dorks

Search for filenames, error strings, or config file names unique to the target:

```
filename:install_cmstep1.aspx
filename:web.config "ConnectionString"
<unique error message from the live app>
<internal microservice name>
```

Also search software name + `password`, `token`, `secret`, `api_key`.

Beyond GitHub: Try Sourcegraph (`sourcegraph.com/search`) and `grep.app`, they index repos across GitHub, GitLab, and Bitbucket.

2.6 Free Trials and Sales Calls

Most vendors offer trials. 14 days is enough to pull source files and run a local audit. If a trial requires a sales call, that's a good sign, fewer researchers have ever audited it so undiscovered bugs are more likely.

2.7 Freelancing Platforms

Search Fiverr or Freelancer for people who implement the target software. Many have licensed access to enterprise installation files. Pay them to install it on your server and upload the files.

2.8 Package Registries (npm, pip, etc.)

The published package isn't always the same as the GitHub repo. Pull and inspect:

```
npm pack <package-name>      # downloads .tgz
pip download <package-name>  # downloads wheel/sdist
```

Also useful: pull the version your target uses.

2.9 Chaining Vulnerabilities

LFI, XXE, path traversal, or SSRF can read source files from a live target. On .NET, read DLLs from the `bin` folder and decompile locally. Check the program's policy first because reading source code via bugs may not be in scope.

2.10 Decompilation

Compiled doesn't mean unreadable. Use the right tools:

Java (JAR/WAR/EAR):

Tool	Best for
JD-GUI	Quick visual exploration
jadx	CLI / scripting / bulk
CFR	Modern Java (lambdas, complex switch)
Procyon	Same as CFR
Fernflower	Obfuscated/complex code

java-deobfuscator	Protected code (e.g. Zelix KlassMaster)

IntelliJ IDEA decompiles JARs transparently when you open them.

.NET / **C#:** dotPeek (free), dnSpy (decompiler + debugger), ILSpy (open source, VS Code extension)

Python `.pyc`: `uncompyle6` or `decompile3`

Native binaries: Ghidra (free, NSA-made), IDA Pro (paid, best-in-class)

Output won't be perfect, but it's enough to trace data flows, find sinks, and understand auth logic.

Part 3: Methodology

3.1 Define Scope First

> **TLDR:** Pre-auth bugs = higher value, higher CVSS, better payouts.

Decide before reading anything: pre-auth only, or post-auth too?

Pre-auth vulnerabilities require no account and scale better across programs. Shubham Shah focuses almost exclusively on pre-auth surface. Post-auth bugs are worth it mainly if they lead to an auth bypass.

First thing to do in any new codebase: Find every route with no auth middleware. Those are your targets.

3.2 Mapping Attack Surface

> **TLDR:** Know every entry point before picking one to dig into. Routes, services, ports, WebSockets, all of it.

Step 1: Find all running services and ports

Before reading routes, know what's running:

- `docker-compose.yml` or Kubernetes manifests → internal services

- EXPOSE lines in Dockerfiles
- `ss -tlnp` or `netstat -tlnp` on a local instance
- nginx/Caddy/HAProxy configs → what's proxied internally

Internal services (Redis, Prometheus exporters, admin panels, image renderers) are often reachable via SSRF even if not publicly exposed. Map them now.

Step 2: Find the routing file

| Framework | Where routes live | | | | Java Servlets | `WEB-INF/web.xml` | | Spring | `@RequestMapping`, `@GetMapping` on controllers | | Rails | `config/routes.rb` | | Django | `urls.py` | | Express | `app.js`, `routes/` dir | | Go | `api.go`, `main.go`, `http.HandleFunc()` | | ASP.NET | `[Route]`, `[HttpGet]` attributes; `Startup.cs` | | Laravel | `routes/web.php`, `routes/api.php` | | Flask | `@app.route()` decorators |

Step 3: Annotate each route with its auth requirement

No middleware = highest priority:

```
Route                | Method | Auth Middleware | Priority
|--|--|-
/api/users/:id       | GET    | requiresAuth    | low
/avatar/:hash        | GET    | none            | HIGH ←
/api/reset-password  | POST   | none            | HIGH ←
```

This is exactly how Rhyno found the Grafana SSRF: `/avatar/:hash` was the only route without `reqSignedIn`.

Step 4: Look beyond HTTP

Auth middleware only covers HTTP routes. Also map:

- WebSocket endpoints and their message handlers
- GraphQL - which queries/mutations need auth?
- gRPC `.proto` files - which RPCs require auth metadata?
- Background job queues: can an attacker enqueue a job?
- File upload handlers outside the main auth flow
- OAuth/OIDC callbacks are always unauthenticated by design. Check the handler for missing `state` validation, unvalidated `next`/`redirect_uri` param, and reusable authorization codes.

Step 5: For post-auth bugs, sort by privilege level

Routes any authenticated user can hit are more impactful than admin-only ones. Look for privilege escalation paths, lower-privilege users reaching higher-privilege routes.

3.3 Sniff for Blood

> **TLDR:** When something looks off, chase it.

When something behaves unexpectedly, stop and understand why before moving to the next file. Unexpected things almost always mean the developer lost control of something.

What "blood" looks like in code:

- An endpoint missing auth when every other one has it
- A URL-fetching call that follows redirects
- A custom sanitizer that doesn't match what the standard library does
- Parameters named `redirect_url`, `next`, `url`, `file`, `path`, `target`
- An XML parser with no entity configuration
- A deserialization call accepting non-internal input
- `// TODO: validate this` or `// temporary, fix later`

CVE-2020-13379: full chain:

1. Route audit: `/avatar/:hash` has no auth. Small anomaly.
2. Read the handler: calls `GoFetch(gravatarSource + hash + "?" + reqParams)`. Hash is attacker-controlled. That's an SSRF seed.
3. Gravatar's `?d=` param redirects to `i0.wp.com/<anything>` when the hash has no image.
4. `i0.wp.com/1.bp.blogspot.com/...` → redirects to blogspot.
5. Test `%3f` (encoded `?`) in the path: `i0.wp.com/test%3f/1.bp.blogspot.com/` → redirects to blogspot.
6. Host a PHP redirect server. Chain: Grafana → Gravatar (`d=`) → i0.wp.com (`%3f` smuggling) → attacker redirect → 169.254.169.254.
7. Bonus: Grafana always returns `Content-Type: image/jpeg` no matter what it fetched. Use it to escalate other blind SSRFs to full-read.

Habits:

- When something looks interesting, open a scratch file and write every question it raises
- Ask “what’s the worst case if I fully control this?” then work backward
- When you see a protection, immediately ask “can this be bypassed?”
- A bug in one endpoint is often present somewhere else too
- If a protection relies on a third-party service, research that service independently

3.4 Dynamic Debugging

> **TLDR:** Run the app locally, attach a debugger, set a breakpoint at the sink you care about, send a request, and see exactly what data arrives there. Faster than tracing it by hand.

Always run the software locally. Setup cost pays off throughout the audit.

General flow:

1. Spin up local instance (Docker, VM, native)
2. Attach debugger to the running process
3. Set a breakpoint at the sink
4. Send a crafted request
5. Inspect the call stack and variable values when execution pauses

Java: JVM has built-in remote debug (JDWP)

```
java -
agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:
5005 -jar app.jar
```

Attach in IntelliJ: **Run → Attach to Process** → port 5005. Set breakpoints in decompiled class files, IntelliJ pauses there.

Node.js: built-in inspector

```
node --inspect app.js
```

```
node --inspect-brk app.js  # pause immediately on start
```

Open `chrome://inspect` → click “inspect”. Or attach from VS Code:

```
{ "type": "node", "request": "attach", "port": 9229 }
```

Python: `pdb` needs no setup

```
breakpoint()  # Python 3.7+
```

For web frameworks (Flask, Django, FastAPI), use `debugpy`

```
pip install debugpy
python -m debugpy --listen 5678 --wait-for-client app.py
```

VS Code: `{ "type": "python", "request": "attach", "connect": { "host": "localhost", "port": 5678 } }`

PHP: Xdebug in `php.ini`

```
zend_extension=xdebug.so
xdebug.mode=debug
xdebug.start_with_request=yes
xdebug.client_host=127.0.0.1
xdebug.client_port=9003
```

Install “PHP Debug” in VS Code. Any request to the app gets caught at the next breakpoint.

.NET / **C# - dnSpy**: decompiles and debugs without original source.

1. Open DLL/EXE in dnSpy
2. **Debug → Attach to Process**
3. Set breakpoint in the decompiled view
4. Send request

For Docker: expose port 4024 (`vsdbg`) and use JetBrains Rider to remote debug.

Go - **Delve** (`dlv`)

```
go install github.com/go-delve/delve/cmd/dlv@latest
dlv debug --headless --listen=:2345 --api-version=2 ./cmd/
server
```

VS Code: `{ "type": "go", "request": "attach", "mode": "remote", "port": 2345 }`

Ruby: `binding.pry` (with `pry` gem) or `binding.irb`.

For VS Code:

```
rdbg --open --port 12345 -- bundle exec rails server
```

3.5 Tracking Code Flows with VS Code Bookmarks

> **TLDR:** Use labeled bookmarks to mark SOURCE → PROPAGATOR → SANITIZER → SINK as you trace.

Tracing a vulnerability through dozens of files means losing your thread every time you navigate away, you can fix this with bookmarks.

Extension: *vscode-numbered-bookmarks* by Alessandro Fragnani

Use labeled bookmarks to describe what each point is:

```
Ctrl+Shift+P → "Bookmarks: Toggle Labeled"

"SOURCE: user input enters via req.body.email"
"PROPAGATOR: value assigned to query variable"
"SANITIZER?: strip_tags called, is this effective?"
"SINK: SQL execute() called here"
```

Workflow:

1. Bookmark the route handler: `SOURCE: entry point`
2. Follow data with `F12` (Go to Definition) and `Alt+Left` (Go Back)
3. Bookmark each function boundary data passes through
4. Bookmark any sanitizer, note if it's actually effective
5. Bookmark the sink

Now the Bookmarks panel is your complete flow map. Jump between SOURCE and SINK instantly. Reconstruct the chain in the next session without re-tracing.

For multiple flows, use prefixes:

```
[FLOW-1] SOURCE: avatar hash from URL path
[FLOW-1] PROPAGATOR: hash passed to GoFetch()
[FLOW-1] SINK: GoFetch makes outbound HTTP request
[FLOW-2] SOURCE: email from reset form
[FLOW-2] SINK: SQL query executed
```

Export your map: "Bookmarks: List from All Files" → copy to notes. You get file paths + line numbers for everything.

Pair with TODO Highlight for inline questions while reading:

```
//TODO: does url_decode run before or after this sanitizer?
//TODO: is this route reachable without auth?
```

Bookmarks = jump points across the codebase. TODOs = inline questions. Use both.

GitLens: It shows inline blame, full file history, and commit diffs right in VS Code. Especially useful for diff-based review because you can see exactly when a security-relevant line was introduced and what the commit message said.

3.6 JxScout

> **TLDR:** JxScout plugs into your proxy (Burp/Caido), collects every JS file your target loads, beautifies it, pulls hidden Webpack chunks you'd never trigger manually, reverses source maps, and dumps it all into a folder you can open in your IDE.

What it gives you:

- **Beautified JS** - every captured file is auto-formatted with Prettier so you can actually read it
- **Hidden chunks** - Webpack/Vite apps lazy-load numbered chunks, JxScout brute-forces chunk IDs to pull modules you'd never trigger by browsing, like admin panels or feature-flagged flows
- **Source map reversal** - automatically tries to fetch `.map` files for every JS it captures. When it hits, you get original TypeScript/JSX with real variable names
- **AST analysis** - the VS Code extension parses the output and flags API endpoints, auth logic, and config values so you have starting points for manual review

We recently had Francisco doing a masterclass on how to use JxScout and he gave us some really useful tips. If you're interested in that, check our subscriptions on our Discord server.

3.7 AI + Source Code: What's Actually Possible

> **TLDR:** What AI can do for you depends entirely on what code you have. Full source code and frontend-only JS are two very different starting points.

When you have full source code (open source, decompiled, Docker pull, etc)

This is where AI is most useful, because you can feed it the actual backend logic and it can reason about things that would take hours to trace manually:

- **Taint analysis:** "Does user input from this endpoint reach a SQL query without sanitization?" AI can trace multi-file data flows that static analyzers miss, especially through stuff like middleware chains, decorators and dependency injection
- **Custom sanitizer auditing:** paste a sanitizer and ask "how do I bypass this?" AI is genuinely good at spotting incomplete blocklists, case sensitivity issues, and encoding gaps
- **Auth model mapping:** "Which routes in this codebase have no authentication middleware?" Feed it the routing config and middleware setup, get back a list sorted by exposure
- **Diffing patches:** give it a CVE patch and the surrounding code, ask "where else does this same pattern exist?" Faster than grepping when the pattern isn't a simple string match
- **Understanding internals:** "How does this framework's deserialization work?

What types can I instantiate?" AI knows most major frameworks well enough to explain the plumbing you'd otherwise have to read source for

- **Decompiled code translation:** decompiler output (jadx, dnSpy, Ghidra) is often ugly and missing context. AI can clean it up, rename variables to something meaningful and explain what a method actually does

When you only have frontend JavaScript (web targets, no backend access)

You have less to work with, but AI still accelerates the workflow significantly:

- **API endpoint extraction:** feed it beautified JS bundles (from JxScout or manual pulls) and ask for every API call, endpoint path, and parameter name.
- **Hidden params:** JS bundles contain parameter names the UI doesn't expose. AI can pull every key name from request builders, form serializers, and config objects.
- **Auth flow reconstruction:** "How does this app handle login, token refresh, and session storage?" AI can read the auth module and map the full flow, including where tokens are stored and how they're attached to requests
- **Business logic mapping:** "What roles exist in this app and what can each one do?" Frontend code often has role checks, permission constants, and feature flags that reveal the backend's authorization model
- **Identifying dead or debug code:** AI can spot conditional blocks gated on `isDev`, `isStaging`, `enableDebug` flags. These features sometimes remain accessible in production if the flag is client-side only

Both scenarios

- **Generating PoCs:** once you've found a promising flow, AI can draft the exploit script: build the HTTP request, handle encoding, chain the steps.
- **Rubber ducking?:** describe what you're seeing and what feels off. AI is a very good second pair of eyes(or a nose) for the "this stinks but I don't know why"

3.8 Other Automated Tools

> **TLDR:** Use tools to narrow down what's worth working on manually.

Semgrep: pattern matching. Finds things that *look* wrong. Can't trace data flow across function boundaries. Good for an initial sweep. Use the `elttam` Java ruleset.

CodeQL: AST-based taint analysis. Actually asks "does attacker data reach this sink?"

Much more powerful. Great for Java and C#. No PHP support.

Snyk: free tier, VS Code plugin, GitHub integration. Covers both vulnerable dependencies and insecure code patterns in one pass. Lower barrier to entry than CodeQL, good for a quick first pass on a new target.

AI-assisted tools: traditional static analyzers catch under 20% of security issues. Modern AI tools (using NLP and AST analysis to understand code intent, not just pattern-match) reach detection rates of 42–48%. Worth layering in on top of semgrep/CodeQL, especially for complex codebases where context matters.

Recommendation: Semgrep first for an initial list, then validate manually. For Java/C#, invest in CodeQL for real taint tracking. Use AI tools as an additional layer, not a replacement for any of the above.

Fuzzing: if the target parses a file format, archive, or custom protocol, run a fuzzer at the parser. Static analysis tells you what the code *should* do while fuzzing tells you what it does with input it wasn't designed for.

Tools by language:

- **AFL++** — C/C++ native binaries
- **Jazzer** — Java (JVM bytecode)
- **Atheris** — Python
- **Fuzzilli** — JavaScript engines

Especially worth running when you see custom deserialization, file upload handlers, or anything that accepts binary/structured data from users.

3.9 Dependency Auditing

> **TLDR:** Third-party libraries are sinks too. Don't assume they're safe.

If the app passes user input to ImageMagick, ExifTool, or any external binary, that binary inherits the attack surface.

- Check every dependency version against CVE databases
- For libraries processing user data (image processors, XML parsers, PDF generators), read their source for edge cases
- Java's `DocumentBuilder` expands external entities by default. The developer

must add two lines to disable it. The missing config *is* the vulnerability.

3.10 Don’t Give Up Too Early + Take good notes (again)

Shubs focuses on a target for at least 1~2 weeks before writing off a target. Most hunters quit after a few hours but the bugs are usually there, they just haven't understood the app well enough yet. Before ending each session, dump every interesting spot you found to a file, even things you couldn't exploit. Start the next session from that list.

And always remember that the better you are at taking notes for yourself, the easier it will be for your AI agents to go through those notes and help you later! =)

3.11 Common Patterns Across Top Write-ups

> **TLDR:** Recurring patterns we see across write-ups from top researchers. Each one represents a way of thinking about code that consistently leads to higher severity bugs.

Patch diffing: When a vuln gets patched, reading the diff tells you exactly what the developers thought the problem was. But you'll often find the same problem in other areas of their code.

Parser differentials: When a security layer and the backend parse the same input using different logic, they can disagree on what the data actually is. The security layer sees something safe, the backend interprets it differently. This is a classic that we keep seeing over and over.

Pre-auth surface: Anything reachable without authentication is reachable by anyone on the internet. Even if there are only a few unauthenticated endpoints, those are the ones worth spending the most time on. This leads to unauthenticated RCE, SSRF, and information disclosure at scale.

Chaining: A bug that is a low on its own can become a crit when combined with another. For example, an auth bypass that gives you access to an internal endpoint + injection in that endpoint = pre-auth RCE.

Validate-then-transform: When data is validated or sanitized and then modified afterward (decoded, normalized, concatenated), the modification can undo the security check. The data that was "safe" at check time is no longer safe when it executes.

Fail-open defaults: When a security check encounters an error (malformed input, unreachable auth service, unexpected type), does it block the request or let it through? Many implementations default to allowing the request when something unexpected happens.

Alternative paths: New security controls often get added to the main request path, but legacy endpoints, debug routes, internal APIs, or older protocol handlers still reach the same backend without going through those controls.

Incomplete fix: When a vendor patches a reported vulnerability, the fix sometimes only addresses the specific PoC rather than the underlying root cause. The same vulnerable pattern may exist in other handlers, or the fix can be bypassed with a slight variation.

Part 4: Vulnerable Code Patterns & Techniques

> **TLDR:** Patterns to spot while reading code, plus techniques and real-world examples. None of these are guaranteed bugs but each one is a signal to slow down and look closer.

4.1 Sanitization Followed by Data Modification

Sanitize input, then transform it in a way that undoes the sanitization.

```
$input = sanitize_html($user_input);  // strips HTML
$input = urldecode($input);           // %3Cscript%3E becomes
<script>
echo $input;                          // XSS
```

Fix: Decode first, then sanitize. Order matters.

4.2 Auth Check Inside an If Statement

Auth check placed inside a conditional body where the condition is user-controllable.

```
function process_request($action) {
    if ($action === "admin_only_action") {
        check_admin_privilege();  // skipped for all other
values
        do_admin_thing();
    }
    do_general_thing();  // always runs
}
```

The correct pattern exits immediately:

```
if (!is_authenticated()) {
    return unauthorized();  // stops here
}
do_thing();
```

Look for: security checks inside an `if` where you control the condition.

4.3 Security Check with No Control Flow Effect

The check runs, detects a problem, but doesn't stop execution.

```
if (!is_valid_input($data)) {
    $error = true;  // set but never checked
}
execute_query($data);  // runs anyway
```

Look for: security checks that don't `return`, `exit`, `throw`, or `die`. If they only set a variable, trace whether anything downstream uses it in time.

4.4 Bad Regex

Regex used for security validation is very often wrong. Test any security-related regex on *regex101.com* with the correct language.

Unescaped dot: `.` matches any character. `example.com` also matches `exampleXcom`. Write `example\.com`.

Missing anchors: `/example\.com/` matches `evil.com?to=example.com`. Add `^` and `$` anchors.

Multiline: `^` and `$` behavior changes with the multiline flag. Smuggling on a second line may work if the flag is missing or set incorrectly.

Case: without `/i`, `SELECT` doesn't catch `select`.

- **vs `+`:** matches zero or more. If you need at least one, is wrong. Use `+`.

JS `.replace()` first-match-only:

```
"aaa".replace("a", "b")    // "baa" = only first replaced!
"aaa".replace(/a/g, "b")   // "bbb" = needs /g flag
```

```
"aaa".replaceAll("a", "b") // "bbb" = or use replaceAll
```

JS capture groups in replacement: `String.replace()` supports `$&`, `$'`, `$<n>` in the replacement string. If user controls the replacement argument, they can reinject filtered characters.

4.5 Non-Recursive Replace Sanitization

Stripping a sequence like `../` only once instead of looping.

```
Input:  ..../
After one pass of removing '../':  ../    ← path traversal
```

The attacker sandwiches the blocked sequence inside itself. One pass collapses it back.

Fix: Loop until no change, or use `os.path.abspath()` + verify the result is inside the expected directory.

4.6 Dynamic Function Calls

A string variable (potentially attacker-controlled) used to call a function or method.

```
$method = $_GET['action'];
$this->$method($_REQUEST);  // calls any public method with the
full request
```

```
String methodName = request.getParameter("action");
Method m = this.getClass().getMethod(methodName);
m.invoke(this);  // programmable eval
```

If the method name is attacker-controlled, they can invoke anything on the object.

Look for: `$obj->$var`, `call_user_func`, `call_user_func_array` in PHP; reflection APIs in Java/.NET.

4.7 Type Confusion

App expects one type, attacker sends another, unexpected behavior follows.

GitLab example: Password reset expected `email` as a string. Attacker sent an array:

```
user[email][]=victim@example.com&user[email]
[]=attacker@example.com
```

App validated using the first email but sent the reset link to *both* → ATO

Common scenarios:

- Array where string expected → SQLi, query changes, string checks bypassed
- String where integer expected → type coercion bypasses numeric comparisons
- `null` where non-null expected
- Wrong type causes a crash → low-impact CSRF becomes DoS

Most affected: PHP, Ruby, JavaScript (implicit coercion).

Test: For any ID or email param, try `param[]=value` and `null`. Unexpected errors reveal how the value is used internally.

4.8 Insecure Randomness

Using a non-cryptographic RNG for security-sensitive values (session tokens, reset tokens, CSRF tokens, API keys).

```
// Bad: 48-bit seed, predictable
Random r = new Random();
r.nextBytes(sessionId);

// Good: cryptographically secure
SecureRandom sr = new SecureRandom();
sr.nextBytes(sessionId);
```

An attacker who observes a few outputs from `java.util.Random` can derive the seed and predict all future session IDs.

Look for: `java.util.Random`, `Math.random()` (JS), `rand()` (PHP), `random.random()` (Python). Any function not explicitly documented as cryptographically secure shouldn't touch security values.

4.9 XXE - XML Parsed Without Entity Restrictions

XML parsed without disabling external entity expansion and DTD processing.

```
// Vulnerable: expands entities by default
DocumentBuilderFactory dbf =
DocumentBuilderFactory.newInstance();
Document doc = dbf.newDocumentBuilder().parse(inputStream);

// Safe: explicitly disabled
dbf.setFeature("http://apache.org/xml/features/disallow-
doctype-decl", true);
dbf.setFeature("http://xml.org/sax/features/external-general-
entities", false);
dbf.setFeature("http://xml.org/sax/features/external-parameter-
entities", false);
```

The vulnerability is the *absence* of the defensive lines, the parser is correct but not configured.

Look for: any `DocumentBuilder`, `SAXParser`, `XMLInputFactory`, `JAXBContext`, or `Unmarshaller` call with no defensive config next to it.

4.10 Zip Slip

> **TLDR:** Archive extraction without path normalization lets an attacker write files anywhere on the server.

A crafted ZIP or TAR can contain entries with filenames like `../../etc/cron.d/evil`. If the code extracts without checking the destination path, the file lands outside the intended directory, leading to arbitrary file write, which often chains into RCE.

```
// Vulnerable
ZipEntry entry = zip.getNextEntry();
File output = new File(destDir, entry.getName()); //
entry.getName() is attacker-controlled
output.getParentFile().mkdirs();
Files.copy(zip, output.toPath());
```

```
// Safe: verify the resolved path stays inside destDir
File output = new File(destDir,
entry.getName()).getCanonicalFile();
if (!
```

```
output.toPath().startsWith(destDir.getCanonicalFile().toPath())
) {
    throw new SecurityException("Zip Slip detected");
}
```

Look for `entry.getName()`, `tarEntry.getName()`, `zipEntry.getName()` passed directly to a `File` or path constructor without a canonical path check afterward. Common in file upload handlers, plugin installers and anything that processes user-submitted archives.

4.11 CVE-2020-13379 (Grafana Unauthenticated SSRF)

> **TLDR:** Route audit → unauthenticated endpoint → server-side URL fetch → redirect chain → full SSRF → 25+ crits

1. **Scope:** Unauthenticated endpoints only.
2. **Route audit:** Read `pkg/api/api.go`. Every route has `reqSignedIn` except one: `/avatar/:hash`.
3. **Read the handler:** Calls `GoFetch(gravatarSource + hash + "?" + reqParams)`. Hash = attacker-controlled. Server fetches a URL built from user input.
4. **Research Gravatar:** `?d=` redirects to `i0.wp.com/<anything>` when hash has no image.
5. **Research `i0.wp.com`:** `i0.wp.com/1.bp.blogspot.com/...` redirects to blogspot. Test: `i0.wp.com/test%3f/1.bp.blogspot.com/` → redirects to blogspot.
6. **Chain it:** Host redirect server at `redirect.rhynorater.com`. Chain: Grafana → Gravatar → i0.wp.com (`%3f` smuggling) → redirect server → `169.254.169.254`.

Final payload:

```
/avatar/
test%3fd%3dredirect.rhynorater.com%25253f%253b%252fbp.blogspot.
com%252f169.254.169.254
```

Result: 25+ crits, 15+ highs.

4.12 SSRF Pivoting

Once you have SSRF, the next step is figuring out what's actually running internally. Check `docker-compose.yml`, Kubernetes manifests, and service configs to find internal service names and ports you can now reach.

Cloud metadata:

```
169.254.169.254/latest/meta-data/iam/security-credentials/
<ROLE>  → AWS IAM creds
169.254.169.254/latest/user-data
→ often contains secrets
```

Internal services:

Every target is different, the goal is to read the source code, find what's running internally, and look up how to interact with it. Some common ones you'll encounter:

- Grafana Image Renderer: `localhost:3001/render?url=<your-url>` → renders arbitrary HTML → JS execution in headless browser
- Prometheus Redis Exporter: `localhost:9121/scrape?target=redis://127.0.0.1:7001&check-keys=*` → dumps all Redis keys

4.13 Bypassing Protections by Chaining

Finding a protection doesn't mean the vulnerability class is closed, just that you need to chain something else to bypass it.

Some examples worth trying:

- **SSRF with URL allowlist** → find an open redirect on an allowed domain
- **XSS with CSP** → find a JSONP endpoint on an allowed domain, or use `blob:` / `data:` bypass
- **SQLi with WAF** → encoding, comments, alternative syntax
- **Path traversal with normalization** → double encoding, backslashes, null bytes

If you hit a whitelist or any other partial protection, look for a way around it.

4.14 Config File Parsing Bugs

Config file parsers have quirks that can be exploited:

- **Length/truncation limits:** inih (C library) at **200 bytes** (`INI_MAX_LINE`) returns a non-zero error code on overlong lines, which callers often ignore; PAM `pam_group` at **1000 bytes** (`PAM_GROUP_BUFLEN`); legacy BSD syslog (RFC 3164, now obsoleted by RFC 5424) at **1024 bytes** total. The dangerous pattern is `fgets()`-based C parsers: when a line exceeds the buffer, unconsumed bytes remain in the stream and the next `fgets()` call reads them as a new line, content past the boundary can be evaluated as a fresh directive. PHP's `parse_ini_file` has no line limit but silently truncates values at an unquoted `;` in `INI_SCANNER_NORMAL` mode (e.g. `password=secret;comment` → `password=secret`).
- **Duplicate sections:** Some parsers let a second section definition overwrite the first. Inject a second occurrence of a critical section to control the config.
- **Whitespace/encoding:** Tabs vs. spaces, `\r\n` vs `\n`, Unicode whitespace. Parsers handle these differently and that can break the expected structure.

Part 5: Bug Bounty Specifics

5.1 Zero-Days in Deployed Software

When a bug affects software used by many programs, check each program's policy before mass exploitation. Some cover third-party software they deploy, others don't.

5.2 Responsible Disclosure

For bugs found outside a formal bug bounty program:

1. Email the vendor's security team (not a bug bounty platform)
2. Give 90 days (Google Project Zero standard)
3. If they patch within 90 days, give 30 more days grace
4. Disclose publicly after, with or without a PoC

5.3 Mass Exploitation Setup

When one bug affects many programs:

- Recon pipeline to find all affected targets before public disclosure
- Report templates ready to customize (`github.com/rhynorater/reports`)
- Trusted collaborators to split targets and file simultaneously
- Splitting agreements set up in advance

5.4 Program Selection

Top 10–25% of programs pay 90% of bounties. But smaller programs (under $10K/90 days) have less competition and room to become a domain expert. Both strategies are valid.

Extra tips from *Shubs*:

I've spent a lot of time going through more basic tips in previous videos, so I'll impart whatever wisdom I've learnt since those previous videos. If you want to check those out, you can look at Code Review the Offensive Security Way. Since that video, here are my top 5 new tips:

1. Instrument the application deeply. If you look at our Magento CosmicString vulnerability on the Assetnote blog, you'll find that it was a complex nested deserialization bug. We spent a lot of time setting up debuggers, but actually it was way easier to understand the flow by just putting in some strategic prints in the code and reading outputs. A debugger was helpful, but printing was faster and easier to grok. (*https://www.assetnote.io/resources/research/why-nested-deserialization-is-harmful-magento-xxe-cve-2024-34102/*)
2. Debuggers are usually non negotiable. Whether it's PHP, JavaScript, a .NET product, or Java, putting the effort into getting a sandbox environment with a debugger attached is always worth it. For C#, I highly recommend you use JetBrains Rider, it makes the debugging process so easy and fun. When we were auditing Sitecore, a debugger was absolutely key in proving an order of operations bug, and when we audited DotNetNuke, we needed to see the unicode transformation happen. See the research here: *https://www.assetnote.io/resources/research/leveraging-an-order-of-operations-bug-to-achieve-rce-in-sitecore-8-x---10-x/* & *https://slcyber.io/research-center/abusing-windows-net-quirks-and-unicode-normalization-to-exploit-dnn-dotnetnuke/*
3. Understand dynamic code evaluation paths. This is something we've historically had a lot of luck with at Assetnote. Many complex products support the evaluation of some sort of templating language, or in some cases as simple as XSLT transformation in Java, RhinoScript, or Jython, or Java Expression Language. Sandbox bypasses are also a lot of fun and usually worth a long period of investigation. See: *https://slcyber.io/research-center/finding-critical-bugs-in-adobe-experience-manager/*
4. Don't resist AI, instead equip your agent with everything it needs to be successful. This doesn't mean you stop manually auditing. You need to feed AI hunches, and you need to double down in specific areas when it doesn't think something is possible, but you do. The source code auditing ecosystem is changing rapidly with capable models like Opus 4.6. We have to continue to learn and adapt with every tool we have available.
5. Make sure the code you are focusing on, is actually the code you want to be focusing on. Speed is critical in auditing code, so when looking at large enterprise

bundles, you want to minimise the noise of vendor dependencies as much as possible. We created Hyoketsu at Assetnote for the community, and the project is available on GitHub for free: *https://github.com/assetnote/hyoketsu* - this will save an immense amount of time decompiling large enterprise products.

Quick-Reference Checklist

Before You Start

- [ ] Get source code (Docker Hub, marketplace, GitHub dorks, decompilation, source maps, `.git`)
- [ ] Local instance running for dynamic debugging
- [ ] Scope defined: pre-auth only or post-auth too?
- [ ] Sink list built for this language/framework

First Pass

- [ ] Routes mapped with auth middleware annotated
- [ ] Unauthenticated endpoints highlighted
- [ ] Sources listed (params, headers, cookies, uploads, WebSocket)
- [ ] Custom security functions flagged for later
- [ ] Hardcoded credentials and secrets scanned
- [ ] Config files and dependencies checked

Source/Sink Analysis

- [ ] Sink inventory built
- [ ] Each sink: can user input reach it?
- [ ] Each source: where does it end up?
- [ ] Middleware applied consistently across all routes?

Common Patterns

- [ ] Patch diffing: is the root cause fully addressed?
- [ ] Parser differentials: do security layer and backend parse the same way?
- [ ] Pre-auth surface mapped
- [ ] Chaining: can two low-severity findings combine?
- [ ] Validate-then-transform: data modified after security check?

- [ ] Fail-open: what happens when a check errors out?
- [ ] Alternative paths: do legacy/debug routes bypass new controls?

Vulnerable Patterns

- [ ] Custom sanitizers: case-insensitive? global? recursive? complete?
- [ ] Sanitize then decode?
- [ ] Auth inside a user-controllable if-body?
- [ ] Security check with no bailout?
- [ ] Regex: unescaped dots, missing anchors, wrong flags?
- [ ] JS `.replace()` without global flag?
- [ ] Replace sanitization single-pass?
- [ ] Dynamic function calls from user input?
- [ ] Type confusion: arrays, null, wrong types?
- [ ] CSPRNG for all security-sensitive random values?
- [ ] XML parsing: external entities and DTDs disabled?

Dependencies

- [ ] Versions checked against CVEs
- [ ] Source read for libraries processing user data
- [ ] No assumptions about standard libraries

SSRF

- [ ] All server-side URL-fetching identified
- [ ] Allowlists bypassable via open redirect?
- [ ] Internal services and ports enumerated
- [ ] Cloud metadata reachable?
- [ ] Response content and Content-Type noted

Notes

- [ ] Attack vectors dumped to markdown before ending session
- [ ] Interesting-but-unexploitable spots noted for next session

Tools

- [ ] Semgrep, CodeQL, regex101.com
- [ ] GAU, trufflehog, gitleaks
- [ ] JD-GUI/jadx, dotPeek/dnSpy
- [ ] git-dumper/GitTools, dive, restore-source-tree

Resources/worth reading and watching:

- *https://youtu.be/48vMaDUv530?si=XmBfAH7f-Ceqvibu*
- *https://www.youtube.com/watch?v=hwIFVIJnlJU*
- *https://youtu.be/weFl9bDnRiA?si=j8iv8uJzDHQS-6TY*
- *https://youtu.be/p91UpSPfv1Q?si=ux2mD71vEWKYZlmR*
- *https://youtu.be/gvrZbOQrAgc?si=gLaEIuQ65LRa6IeQ*
- *https://youtu.be/4A13Mwjf_Jg?si=I4QPfhPGOKb0K6gb*
- *https://github.com/benhoyt/inih/blob/master/README.md*
- *https://github.com/linux-pam/linux-pam/blob/master/modules/pam_group/pam_group.c*
- *https://www.rfc-editor.org/rfc/rfc3164.html*
- *https://wiki.sei.cmu.edu/confluence/spaces/c/pages/87152445/FIO20-C.+Avoid+unintentional+truncation+when+using+fgets+or+fgetws*
- *https://php.watch/articles/parse_ini_string-file-security-considerations*
- *https://security.snyk.io/vuln/SNYK-JS-TOTALJS-1077069*
- *https://lab.ctbb.show/research/totaljs-rce-gadgets*
- *https://docs.ruby-lang.org/en/2.0.0/security_rdoc.html*
- *https://www.veracode.com/blog/secure-development/spring-view-manipulation-vulnerability/*
- *https://www.yeswehack.com/learn-bug-bounty/open-source-guide-code-analysis*

The more you know about an app, the easier it becomes to find some bugs. *Get intimate with it.*

- *Special thanks to @rafax00, @apostolovd and @infosec_au for taking some time to review it and helping me improve this script!*
- *Special thanks to Autumn Buggs for making the graphics and allowing me to use them on this week's HackerNotes =)*

Keep Hacking!

— Yuji @ Critical Thinking Team